

Florence: Architecture - LLM Chatbot Service

Overview

The Florence Large Language Model Chatbot Service is a dedicated microservice built with FastAPI and Python 3.13. It provides an artificial intelligence conversational agent to assist patients with chronic disease management. The system relies entirely on the external Florence Data Service to validate authentication tokens and retrieve clinical contexts.

Architecture

The application follows a modular design separating routing, business logic and data aggregation. It uses LangChain to orchestrate prompts and handle communication with external language models. The service does not connect directly to the database. Instead, it executes HTTP requests against the core data service to access user profiles, biometric measurements, dietary records and conversation history.

Environment Configuration

The developer must supply specific environment variables to execute the service locally or in production.

- `LLM_API_KEY` : The authentication key for the selected language model provider.
- `LLM_BASE_URL` : The provider endpoint.
- `LLM_MODEL` : The specific model identifier to utilise.
- `LLM_TEMPERATURE` : The generation parameter controlling response variability.
- `DATA_SERVICE_URL` : The uniform resource locator for the primary backend data service.

Directory Structure

The repository contains several key files and directories that govern the chatbot functionality.

- `main.py` : The application gateway that initialises the server and configures the middleware.
 - `config.py` : The environment configuration module.
 - `routers/` : A directory housing the application programming interface endpoints.
 - `services/` : A directory containing business logic for data aggregation, language model interaction and conversation management.
 - `models/` : A directory defining the data validation schemas.
 - `utils/` : A directory containing helper functions including token validation logic.
 - `pyproject.toml` : The configuration file that defines the Python dependencies.
 - `vercel.json` : The deployment configuration file.
-

Package Management with uv

The project relies on a standard Python configuration file but the architecture is optimised for the `uv` package manager. The developer can synchronise the project environment directly from the configuration file. This ensures all required packages such as FastAPI, LangChain and HTTPX are installed rapidly and consistently across different machines.

Installation Instructions

The developer must follow these steps to prepare the local environment.

1. Install the `uv` package manager on the host system.
2. Execute the appropriate `uv` command to create a virtual environment and install the dependencies.
3. Populate the local environment file with the required model keys and service addresses.
4. Start the application using the Uvicorn server gateway interface.

API Modules

General

- `GET /` : Root endpoint with service information.
- `GET /health` : Global health check endpoint.

Chat Module

The chat module processes incoming user messages and returns generated responses. It relies on custom helper functions to extract the bearer token from the request header and validate it against the data service.

- `POST /chat/message` : Receives a patient query, compiles their clinical context, queries the language model and returns the generated response.
- `GET /chat/history` : Retrieves the chronological list of prior messages for the authenticated patient.
- `DELETE /chat/history` : Permanently removes all stored messages for the authenticated patient.

Health Context Processing

When evaluating a patient message, the system aggregates a comprehensive health context. This context includes demographic information, recent biometric measurements, physical activity logs, dietary records, specific health thresholds, active medications and medical diagnoses.

The system parses these records into a formatted prompt. The language model uses this information to provide personalised guidance while adhering to strict medical guardrails. The instructions explicitly forbid the model from diagnosing conditions or recommending medication dosage changes.